

REC-CIS

[Show one page at a time](#)[Finish review](#)**Duration** 30 mins 37 secs**Question 1**

Correct

Marked out of
1.00[Flag
question](#)

A binary number is a combination of 1s and 0s. Its n^{th} least significant digit is the n^{th} digit starting from the right starting with 1. Given a decimal number, convert it to least significant digit.

Example

number = 23

- Convert the decimal number 23 to binary number: $23^{10} = 2^4 + 2^2 + 2^1 + 2^0 = (10111)_2$.
- The value of the 4th index from the right in the binary representation is 0.

Function Description

Complete the function fourthBit in the editor below.

fourthBit has the following parameter(s):

int number: a decimal integer

Returns:

int: an integer 0 or 1 matching the 4th least significant digit in the binary representation of number.

Constraints

$0 \leq \text{number} < 2^{31}$

REC-CIS

Answer: (penalty regime: 0 %)

Reset answer

```
1 */  
2  * Complete the 'fourthBit' function below.  
3  *  
4  * The function is expected to return an INTEGER.  
5  * The function accepts INTEGER number as parameter.  
6  */  
7  
8 int fourthBit(int number)  
9 {  
10     int binary[32];  
11     int i=0;  
12     while(number>0)  
13     {  
14         binary[i]=number%2;  
15         number/=2;  
16         i++;  
17     }  
18     if(i>=4)  
19     {  
20         return binary[3];  
21     }  
22     else return 0;  
23  
24 }
```

	Test	Expected	Got	
✓	printf("%d", fourthBit(32))	0	0	✓
✓	printf("%d", fourthBit(77))	1	1	✓

Passed all tests! ✓

Question 2

Correct

Marked out of
1.00[Flag
question](#)

Determine the factors of a number (i.e., all positive integer values that evenly divide into a number) and then return the p^{th} element of the list, sorted ascending. If the

Example $n = 20$ $p = 3$

The factors of 20 in ascending order are {1, 2, 4, 5, 10, 20}. Using 1-based indexing, if $p = 3$, then 4 is returned. If $p > 6$, 0 would be returned.

Function Description

Complete the function `pthFactor` in the editor below.

`pthFactor` has the following parameter(s):

`int n`: the integer whose factors are to be found

`int p`: the index of the factor to be returned

Returns:

`int`: the long integer value of the p^{th} integer factor of `n` or, if there is no factor at that index, then 0 is returned

Constraints

$$1 \leq n \leq 10^{15}$$

$$1 \leq p \leq 10^9$$

REC-CIS

```
1 /*
2  * Complete the 'pthFactor' function below.
3  *
4  * The function is expected to return a LONG_INTEGER.
5  * The function accepts following parameters:
6  * 1. LONG_INTEGER n
7  * 2. LONG_INTEGER p
8  */
9
10 long pthFactor(long n, long p)
11 {
12     int count=0;
13     for(long i=1;i<=n;++i)
14     {
15         if(n%i==0)
16         {
17             count++;
18             if(count==p)
19             {
20                 return i;
21             }
22         }
23     }
24     return 0;
25 }
26 }
```

	Test	Expected	Got	
✓	printf("%ld", pthFactor(10, 3))	5	5	✓
✓	printf("%ld", pthFactor(10, 5))	0	0	✓
✓	printf("%ld", pthFactor(1, 1))	1	1	✓

Passed all tests! ✓

REC-CIS

[Show one page at a time](#)[Finish review](#)

Duration 9 mins 49 secs

Question 1

Correct

Marked out of 1.00

[Flag question](#)

You are a bank account hacker. Initially you have 1 rupee in your account, and you want exactly N rupees in your account. You wrote two hacks, first hack can multiply the amount of money you own by 10, while the second can multiply it by 20. These hacks can be used any number of time. Can you achieve the desired amount N using these hacks.

Constraints:

$$1 \leq T \leq 100$$

$$1 \leq N \leq 10^{12}$$

Input

- The test case contains a single integer N .

Output

For each test case, print a single line containing the string "1" if you can make exactly N rupees or "0" otherwise.

SAMPLE INPUT

1

SAMPLE OUTPUT

1

REC-CIS

```

3  *
4  * The function is expected to return an INTEGER.
5  * The function accepts INTEGER n as parameter.
6  */
7
8  int myFunc(int n)
9  {
10     while(n>1)
11     {
12         if(n%20==0)
13         {
14             n/=20;
15         }
16         else if(n%10==0)
17         {
18             n/=10;
19         }
20         else
21         {
22             return 0;
23         }
24     }
25     return 1;
26 }
27

```

	Test	Expected	Got	
✓	printf("%d", myFunc(1))	1	1	✓
✓	printf("%d", myFunc(2))	0	0	✓
✓	printf("%d", myFunc(10))	1	1	✓
✓	printf("%d", myFunc(25))	0	0	✓
✓	printf("%d", myFunc(200))	1	1	✓

Passed all tests ✓

Question 2

Correct

Marked out of 1.00

Flag
question

Find the number of ways that a given integer, X , can be expressed as the sum of the N^{th} powers of unique, natural numbers.

For example, if $X = 13$ and $N = 2$, we have to find all combinations of unique squares adding up to 13. The only solution is $2^2 + 3^2$.

Function Description

Complete the powerSum function in the editor below. It should return an integer that represents the number of possible combinations.

powerSum has the following parameter(s):

X : the integer to sum to

N : the integer power to raise numbers to

Input Format

The first line contains an integer X .

The second line contains an integer N .

Constraints

$$1 \leq X \leq 1000$$

$$2 \leq N \leq 10$$

Output Format

Output a single integer, the number of possible combinations calculated.

REC-CIS

```

3
4  * The function is expected to return an INTEGER.
5  * The function accepts following parameters:
6  * 1. INTEGER x
7  * 2. INTEGER n
8  */
9
10 int powerSum(int x, int m, int n)
11 {
12     if(x==0)
13     {
14         return 1;
15     }
16     if(x<0)
17     {
18         return 0;
19     }
20     int count = 0;
21     for(int i=m;;i++)
22     {
23         int power=1;
24         for(int j=0;j<n;j++)
25         {
26             power*=i;
27         }
28         if(power>x)
29         {
30             break;
31         }
32         count+=powerSum(x-power,i+1,n);
33     }
34     return count;
35 }

```

	Test	Expected	Got	
✓	printf("%d", powerSum(10, 1, 2))	1	1	✓

Passed all test! ✓